

作业 7

——k-HH 数据流算法——

声明：题目来源于 <http://openjudge.cn/>，解题思路参考教授上课讲解内容，代码实现由本人完成。
分享这个文档仅限于学术交流，不用于任何形式的商业用途。

1. 题目

描述

本题为交互题，交互格式与 OpenJudge - 1:估计 p 分位数相同，你的代码需要包含头文件“streaming.h”。

请实现一个数据结构，接受一个 $n \leq 10^6$ 个元素的输入数据流，并在最后输出数据流中出现次数较多的元素，具体要求如下：

你的程序输出中的每一个元素都在输入中出现了至少 $\frac{n}{k} - 0.01n$ 次。输入中每一个出现了至少 $\frac{n}{k}$ 次的元素都必须在输出中出现。你需要实现三个函数：

函数 `void init(int k)`。 k 的意义如前面所示， $k \leq 50$ 。

函数 `void add(int e)` 表示向你的数据结构插入一个新元素 e 。

函数 `std::vector report()`。我们会在输入数据流读入结束时调用该函数，请将满足题目要求的元素放到一个 `vector` 中并返回。

交互库中有一个长为 5000 的数组 `arr`，初始时全部为 0，你可以使用如下两个函数：

`void Set (int a, int b)`。表示将 `arr[a]` 设置为 b ，你需要保证 $0 \leq a < 5000$ 。

`int Get (int a)`。表示询问 `arr[a]` 的值，你需要保证 $0 \leq a < 5000$ 。

你不可以使用任何全局变量向 `report` 传递任何信息，否则本题将获得 0 分。

如下是一个错误的做法：在 `add` 时将 e 记录到自己声明的数组里，并在 `report` 中访问该数组。

输入

第一行两个正整数 n, k ，接下来一行 n 个正整数。

注意：这里描述的是链接后的完整程序的输入/输出格式，仅供测试用途。你提交的程序只能通过 `query` 函数访问数据，请勿自行读入！

输出

第一行一个整数 m 表示输出个数，接下来一行 m 个正整数。

注意：这里描述的是链接后的完整程序的输入/输出格式，仅供测试用途。你只需使用 `report` 返回答案即可。

样例输入

```
5 2
1 2 1 2 1
```

样例输出

```
1
1
```

提示

1) 在以下链接下载交互库 grader.cpp, streaming.h, 以及样例代码 example.cpp:

<https://github.com/crasysky/heavy-hitter/tree/main>

2) 你可以使用少量的局部数组（或 set, map）

2. 思路

题目描述比较抽象，我们一点一点来看。首先，题目明确要求，不能存储这些数据，也就是只能看一些数据，而不能获得总体情况。

我们先来介绍一个工具：哈希函数。我们可以使用哈希函数来将元素映射到一个较小的空间中，这样我们就可以在这个较小的空间中进行统计，而不需要存储所有的元素。具体来说，你可以理解成有很多个垃圾桶，我们对垃圾进行分类的时候，每个垃圾都会被扔进一个桶中。显然，某个垃圾桶中的垃圾数量是大于某一种应该扔进这个桶中的垃圾数量。比如，你手上有塑料瓶，纸杯，易拉罐，水果核。你可以按照金属与非金属来分类，那么塑料瓶，纸杯和水果核就会被扔进一个桶中，易拉罐就会被扔进另一个桶中。现在，你再以另一种垃圾分类的方式，比如厨余垃圾和其他，那么纸杯，塑料瓶和易拉罐就会被扔进一个桶中，水果核就会被扔进另一个桶中。我们的算法是：如果想要统计水果核的数目，我们就统计两种方式中，包含水果核的垃圾桶的垃圾总数最小的那个桶，作为输出。

事实上，我们肯定不会只有两个垃圾桶，也不会只有两种不同的分类方式。假设你有足够多的分类方式，以及足够多的垃圾桶，那么这个结果应该是不会太离谱的。这个算法叫做“count-min sketch”。

更严谨地说，对于每一个哈希函数，都有很多个哈希桶。每当我们看到一个元素，我们就把它放到每个哈希函数对应的哈希桶中。对于每个元素，我们可以通过每个哈希函数来找到它所在的哈希桶，然后统计这个哈希桶中的元素数量。由于桶中元素个数必定大于我们目标的元素数量，所以我们要寻找最小值。

3. 代码实现

这道题目的代码相对比较零散，是通过不同的函数实现一些功能构成的。

3.1. 准备工作

头文件，一些需要用到的常量，我们先声明出来，并且标注好可爱的备注，避免自己在将来指着这段代码骂：哪个人写的?! 看都看不懂。

```
#include <bits/stdc++.h> //这真的不是好习惯哈
#include "streaming.h"
using namespace std;

void init(int k);
void add(int e);
vector<int> report();

void Set(int a, int b); // arr[a] = b, 0 <= a <= 5000
int Get(int a);        // 返回 arr[a] 的值

const int M = 10;      // 哈希函数个数
int cnt[M][300];       // Count-Min Sketch
int total = 0;         // 已处理元素总数
int seed[M];           // 每个哈希函数的随机种子
int K;                 // 候选集大小（由 init 传入）
```

3.2. 哈希函数设计

我们需要设计一个哈希函数来将元素映射到一个较小的空间中，这也就是之前提到的垃圾分类的具体方式。我们需要保证这个哈希函数的随机性，以避免哈希冲突过多。

```
int hashFunc(int x, int idx) {
    int t = x ^ seed[idx];
    return ((t % 300) + 300) % 300; // 安全取模，避免负数
}
```

3.3. 全部代码

这道题目的代码如下，注释写的比较详细了。每个函数的作用在前文中已经有比较详细的描述，这里不再赘述啦~

```
#include <bits/stdc++.h>
#include "streaming.h"
using namespace std;

void init(int k);
void add(int e);
vector<int> report();
void Set(int a, int b); // arr[a] = b, 0 <= a <= 5000
int Get(int a); // 返回 arr[a] 的值
const int M = 10; // 哈希函数个数
int cnt[M][300]; // Count-Min Sketch
int total = 0; // 已处理元素总数
int seed[M]; // 每个哈希函数的随机种子
int K; // 候选集大小（由 init 传入）
int hashFunc(int x, int idx) {
    int t = x ^ seed[idx];
    // 安全取模，避免负数
    return ((t % 300) + 300) % 300;
}

void init(int k) {
    K = k;
    total = 0;
    srand(k); // 用 k 作为随机种子，保证可重复
    for (int i = 0; i < M; i++) {
        seed[i] = rand();
        for (int j = 0; j < 300; j++) {
            cnt[i][j] = 0;
        }
    }
    // 清除候选槽位（前 K 个位置）
    for (int i = 0; i < K; i++) {
        Set(i, 0);
    }
}

int count(int x) {
    int res = INT_MAX;
    for (int i = 0; i < M; i++) {
        res = min(res, cnt[i][hashFunc(x, i)]);
    }
    return res;
}

void insert(int x) {
    // 1. 如果已经在候选集中，直接返回
```

```

        for (int i = 0; i < K; i++) {
            if (Get(i) == x) return;
        }
        // 2. 尝试找到空槽位
        for (int i = 0; i < K; i++) {
            if (Get(i) == 0) {
                Set(i, x);
                return;
            }
        }
        // 3. 无空槽：替换当前估计频率最小的候选元素
        int minIdx = -1;
        int minFreq = INT_MAX;
        for (int i = 0; i < K; i++) {
            int freq = count(Get(i));
            if (freq < minFreq) {
                minFreq = freq;
                minIdx = i;
            }
        }
        // 如果当前元素比最小候选更频繁，则替换
        if (count(x) > minFreq) {
            Set(minIdx, x);
        }
    }
    void check() {
        for (int i = 0; i < K; i++) {
            if (Get(i) != 0) {
                int x = Get(i);
                if (count(x) < 1.0 * total / K) {
                    Set(i, 0);
                }
            }
        }
    }
}

void add(int e) {
    total++; // 总计数增加
    check(); // 清理旧候选（基于新阈值）
    for (int i = 0; i < M; i++) {
        cnt[i][hashFunc(e, i)]++; // 更新草图
    }
    if (count(e) >= 1.0 * total / K) {
        insert(e); // 条件插入
    }
}

vector<int> report() {
    check(); // 确保返回的都是当前频繁的元素
    vector<int> res;
    for (int i = 0; i < K; i++) {
        if (Get(i) != 0) {
            res.push_back(Get(i));
        }
    }
    return res;
}

```